

 fsantanna / pico-sdl 



 **Code**  Issues 11  Pull requests 3  Agents  Actions

  main  **pico-sdl** / lua / doc / **api.md** 



fsantanna + pico.id : default layer keys (now returned) ✓

2f95d8e · 8 hours ago



Preview

Code

Blame



Raw



API: pico-lua

Types

In alphabetical order:

- **Anchor:** string | { x: number, y: number }
 - String: 'C', 'NW', 'N', 'NE', 'E', 'SE', 'S', 'SW', 'W'
- **Color:** table | integer | string
 - Table: { ['!'|'%'], r: number, g: number, b: number [, a: number] }
 - Integer: 0xRRGGBB (24-bit, alpha=0xFF) or 0xRRGGBBAA (32-bit)
 - String: 'black', 'white', 'gray', 'silver', 'red', 'green', 'blue', 'yellow', 'cyan', 'magenta', 'orange', 'purple', 'pink', 'brown', 'lime', 'teal', 'navy', 'maroon', 'olive'
- **Dim:** { ['!'|'%'|'#'], [w: number], [h: number] }
 - Missing w or h defaults to 0 ("infer from source").
- **Event:** { tag: string, ... }
 - { tag='quit' }
 - { tag='window.resize', w: integer, h: integer }
 - { tag='key.dn'|'key.up', key: string, ctrl: boolean, shift: boolean, alt: boolean }
 - { tag='mouse.motion'|'mouse.button.dn'|'mouse.button.up', '!', x: integer, y: integer, anchor: Anchor, left: boolean, right: boolean, middle: boolean }
- **Flip:** 'none' | 'horizontal' | 'vertical' | 'both'

- **Mouse:** { ['!'|'%'|'#'], x: number, y: number, anchor: Anchor, left: boolean, right: boolean, middle: boolean }
- **Pos:** { x: number, y: number ['!'|'%'|'#'], anchor: Anchor } }
- **Rect:** { x: number, y: number, [w: number], [h: number] ['!'|'%'|'#'], anchor: Anchor } }
 - Missing w or h defaults to 0 ("infer from source").
- **Rotation:** { angle: integer, anchor: Anchor }
- **Tile:** { w: integer, h: integer }
- **Video:** { dim: Dim, fps: integer, frame: integer, done: boolean }

Operations

In alphabetical order:

- **pico.color**
 - **pico.color.alpha:** Returns a copy of the color with alpha set.
 - `pico.color.alpha (clr: Color, a: integer) -> Color`
 - **pico.color.darker:** Makes a color darker.
 - `pico.color.darker (clr: Color, pct: number) -> Color`
 - **pico.color.lighter:** Makes a color lighter.
 - `pico.color.lighter (clr: Color, pct: number) -> Color`
 - **pico.color.mix:** Mixes two colors.
 - `pico.color.mix (c1: Color, c2: Color) -> Color`
- **pico.cv**
 - Projects a value from L_fr 's frame into L_to 's frame via the hier.up chain. Either layer may be nil (== cur). Each layer must be cur, an ancestor, or a descendant of cur; siblings project via cur in two steps.
 - to accepts two forms:
 - **table:** filled in-place (mode/anchor read from input); returns the same table.
 - **mode string** ('!' , '%' , '#'): builds and returns a fresh value; anchor defaults to 'c' (center; Pos/Rect only).
 - Args are matched left-to-right against the 4-slot template (L_to, to_or_mode, L_fr, fr) . Both explicit nil s and trailing/leading omissions are accepted; a mismatched type leaves the slot empty without consuming the arg, so short forms like (to, fr) , (L_to, to, fr) , (L_to, mode, fr) are all valid. to_or_mode and fr are required.
 - **pico.cv.pos:** Projects a Pos.
 - `pico.cv.pos (L_to: string?, to: Pos, L_fr: string?, fr: Pos) ->`

- Pos
 - `pico.cv.pos (L_to: string?, mode: string, L_fr: string?, fr: Pos) -> Pos`
 - **pico.cv.rect**: Projects a Rect.
 - `pico.cv.rect (L_to: string?, to: Rect, L_fr: string?, fr: Rect) -> Rect`
 - `pico.cv.rect (L_to: string?, mode: string, L_fr: string?, fr: Rect) -> Rect`
 - **pico.cv.dim**: Projects a Dim (no position component).
 - `pico.cv.dim (L_to: string?, to: Dim, L_fr: string?, fr: Dim) -> Dim`
 - `pico.cv.dim (L_to: string?, mode: string, L_fr: string?, fr: Dim) -> Dim`
- **pico.get**: Getters for various properties and settings.
 - **pico.get.effect**: Gets effect configuration.
 - `pico.get.effect () -> { alpha: integer, color: Color, flip: Flip, grid: boolean, rotate: Rotation }`
 - **pico.get.image**: Gets image dimensions.
 - `pico.get.image (path: string) -> Dim`
 - `pico.get.image (mode: string, path: string) -> Dim`
 - `pico.get.image (dim: Dim, path: string) -> Dim`
 - `pico.get.image (dim: nil, path: string) -> Dim`
 - `mode`: '!' pixels, '%' percentage, '#' tiles
 - String-mode and nil/no-arg forms build a fresh `{mode, w=0, h=0}` table; default mode is '!' .
 - Table form is mutated in place; missing `w / h` (zero) axes are filled from the image's natural pixel size, aspect-preserving.
 - Returns the `rel Dim` (the same table when one was passed, or the freshly built one).
 - Forms:
 - `pico.get.image('cat.png')` — pixels
 - `pico.get.image('%', 'cat.png')` — percent
 - `pico.get.image({'%', w=0.5}, 'cat.png')` — fills `h`
 - **pico.get.keyboard**: Gets keyboard modifier state.
 - `pico.get.keyboard () -> { key: string, ctrl: boolean, shift: boolean, alt: boolean }`
 - **pico.get.layer**: Gets current layer name.
 - `pico.get.layer () -> string?`

- Returns `nil` for main layer
- **pico.get.mouse**: Gets mouse position and button state in `layer`'s frame.
`layer` defaults to `cur` when omitted or `nil`.
 - `pico.get.mouse ([layer: string?,] mode: string) -> Mouse`
 - `pico.get.mouse ([layer: string?,] pos: Pos) -> Mouse`
 - `mode`: `'!'` pixels, `'%'` percentage, `'#'` tiles
 - String-mode form defaults `anchor` to `c` (cell-center), matching the Lua-wide default. Use the table form with an explicit `anchor` field to override (e.g. `'NW'`).
 - Returned `Mouse` mirrors `mode` and `anchor` of the request.
 - Forms:
 - `pico.get.mouse('!')` — `cur`, pixels, `C` anchor
 - `pico.get.mouse(nil, '!')` — explicit `nil` = `cur`
 - `pico.get.mouse('world', '%')` — named layer's frame
 - `pico.get.mouse({'!', anchor='NW'})` — explicit anchor
- **pico.get.now**: Gets milliseconds since initialization.
 - `pico.get.now () -> integer`
- **pico.get.pencil**: Gets pencil configuration.
 - `pico.get.pencil () -> { color: Color, font: string?, style: 'fill'|'stroke' }`
- **pico.get.scene**: Gets scene configuration.
 - `pico.get.scene () -> { dim: Dim, tile: Tile, target: Rect, source: Rect, clip: Rect, clear: boolean }`
- **pico.get.text**: Gets text dimensions.
 - `pico.get.text (dim: Dim, text: string) -> Dim`
 - `dim` is **required** and must be a table: `h` is the input font size (in `dim.mode`'s realm).
 - `w` is filled in by the call (when initially `0`).
 - Returns the `rel Dim` (the same table passed in, mutated). To get pixels, use `'!'` mode or multiply by the scene `dim`.
 - No string-mode shorthand: `h` cannot be conveyed.
- **pico.get.video**: Gets video information.
 - `pico.get.video (path: string) -> Video`
 - `pico.get.video (mode: string, path: string) -> Video`
 - `pico.get.video (rect: Rect, path: string) -> Video`
 - `pico.get.video (rect: nil, path: string) -> Video`
 - `mode`: `'!'` pixels, `'%'` percentage, `'#'` tiles

- Returns the full `Video ( {dim, fps, frame, done} )`.
- `Video.dim` **is** the slot-1 input table: caller's `Rect` (table-form, mutated in place) or a freshly built `rect` (string-form / nil / no-arg). Only `mode / w / h` are meaningful for `dim` use; `x / y` may be present (zero by default).
- Missing `w / h` (zero) axes are filled aspect- preserving from the video's natural pixel size.
- **pico.get.window:** Gets window configuration.
 - `pico.get.window () -> { fullscreen: boolean, show: boolean, title: string }`
- **pico.id:** Returns a unique monotonic id.
 - `pico.id () -> integer`
 - `pico.id (prefix: string) -> string`
 - Starts at 1 and is never reset.
 - The prefix form returns `"<prefix>-N"` .
- **pico.init:** Initializes and finalizes pico.
 - `pico.init (on: boolean)`
- **pico.input**
 - **pico.input.delay:** Freezes execution for milliseconds.
 - `pico.input.delay (ms: integer) -> integer`
 - Returns elapsed time in milliseconds (delta time)
 - **pico.input.event:** Waits for an event.
 - `pico.input.event () -> Event, integer`
 - `pico.input.event (ms: integer) -> Event?, integer`
 - `pico.input.event (filter: string) -> Event, integer`
 - `pico.input.event (filter: string, ms: integer) -> Event?, integer`
 - Returns event (or nil on timeout) and elapsed time in ms (delta time)
 - Filters: `'quit'` , `'window.resize'` , `'key.dn'` , `'key.up'` , `'mouse.motion'` , `'mouse.button.dn'` , `'mouse.button.up'`
 - **pico.input.loop:** Blocks on event loop until quit.
 - `pico.input.loop ()`
- **pico.layer:** Creates layers from various constructors.
 - **pico.layer.empty:** Creates an empty layer.

```
pico.layer.empty {
    mode:    string,    -- realm mode (default: '!')
    key:     string,    -- layer name (default: auto '/id-
N')
    up:      string,    -- parent layer (default: detached)
    clear:   boolean,   -- cascade-clear flag, requires up
(default: false)
    dim:     Dim,       -- texture size (default: target w/
h)
    tile:    Tile,      -- tile size in px, dim in TILES
(default: none)
    target:  Rect,      -- scene.target placement (default:
fill parent)
} -> string          -- layer key
```



- **pico.layer.image:** Creates a layer from an image file.

```
pico.layer.image {
    mode:    string,    -- realm mode (default: '!' or '='
when key absent)
    key:     string,    -- layer name (default: path)
    up:      string,    -- parent layer (default: detached)
    path:    string,    -- image file
    target:  Rect,      -- scene.target placement (default:
fill parent)
} -> string          -- layer key
```



- **pico.layer.images:** Creates sub-layer images from a reference "sprite sheet" image.

```
pico.layer.images {
    mode:    string,    -- realm mode (default: '!')
    key:     string,    -- base layer name and sub prefix
(default: path)
    up:      string,    -- parent of the generated sub-
layers
    path:    string,    -- sprite sheet image file
    sheet:   table,     -- grid or explicit spec (below)
} -> {string}         -- generated sub-layer names
```



- sheet :
 - **Grid:** { '#', w: integer, h: integer, [n: integer], [key: string] }
 - generates sub-layers "<key>-01" , "<key>-02" , ...
 - the inner key overrides the outer key prefix
 - uses left-to-right order

- `n` limits the count (default `w*h`)
- **Explicit:** `{ '!', [name1: Rect, name2: Rect, ...] }`
 - generates sub-layers "`<key>-<name>`" for each given `name=crop pair`
- **pico.layer.pixmap:** Creates a layer from a pixmap.

```
pico.layer.pixmap {
    mode:    string,      -- realm mode (default: '!')
    key:     string,      -- layer name (default: auto '/id-
N')
    up:      string,      -- parent layer (default: detached)
    pixels:  {{Color}},   -- pixel matrix (rows of colors)
    target:  Rect,        -- scene.target placement (default:
fill parent)
} -> string              -- layer key
```



- **pico.layer.screenshot:** Creates a layer from a screenshot of a layer.

```
pico.layer.screenshot {
    mode:    string,      -- realm mode (default: '!')
    key:     string,      -- layer name (default: auto '/id-
N')
    up:      string,      -- parent layer (default: detached)
    sup:     string,      -- source layer to capture from
(default: current)
    crop:    Rect,        -- area in sup's frame (default:
full)
    target:  Rect,        -- scene.target placement (default:
fill parent)
} -> string              -- layer key
```



- **pico.layer.sub:** Creates a sub-layer from a source layer.

```
pico.layer.sub {
    mode:    string,      -- realm mode (default: '!')
    key:     string,      -- layer name (default: auto '/id-
N')
    up:      string,      -- parent layer (default: detached)
    sup:     string,      -- source layer to crop from
    crop:    Rect,        -- area in sup's frame (live view)
    target:  Rect,        -- scene.target placement (default:
fill parent)
} -> string              -- layer key
```



- **pico.layer.text:** Creates a layer from text.

```
pico.layer.text {
    mode:    string,    -- realm mode (default: '!!')
    key:     string,    -- layer name (default: auto '/id-
N')
    up:      string,    -- parent layer (default: detached)
    dim:     Dim,       -- h: font height (resolves against
`up`)
    text:    string,    -- text to render
    target:  Rect,      -- scene.target placement (default:
fill parent)
} -> string           -- layer key
```



- uses the current pencil font and color.
- `dim.w` is always inferred from the text
- **pico.layer.video**: Creates a layer from a video file.

```
pico.layer.video {
    mode:    string,    -- realm mode (default: '!!' or '='
when key absent)
    key:     string,    -- layer name (default: path)
    up:      string,    -- parent layer (default: detached)
    path:    string,    -- video file (Y4M format)
    target:  Rect,      -- scene.target placement (default:
fill parent)
} -> string           -- layer key
```



• pico.output

- **pico.output.clear**: Clears screen.
 - `pico.output.clear ()`
- **pico.output.present**: Composites layers and presents buffer.
 - `pico.output.present ([layers: boolean])`
 - `layers`: when `true` (default), composites the window layer hierarchy onto `window.tex` before mirroring to the framebuffer; when `false`, mirrors only (raw, advanced).
 - In non-expert mode `layers` must be `true` (asserted); non-expert auto-presents after each draw op.
- **pico.output.screenshot**: Takes a screenshot of a layer.
 - `pico.output.screenshot (layer: string?, [path: string|rect: Rect,] [rect: Rect]) -> string`
 - `layer`: target layer name; `nil` means current layer.
 - `path`: output file; `nil` (or omitted) uses a timestamped name.
 - `rect`: region in target layer's frame; `nil` (or omitted) captures full. Accepted at position 2 (no path) or 3.

- **Forms:**
 - `pico.output.screenshot(nil)` — cur, default path
 - `pico.output.screenshot(nil, "x.png")` — cur, given path
 - `pico.output.screenshot(nil, rect)` — cur, default path, rect
 - `pico.output.screenshot("L")` — layer L, default path
 - `pico.output.screenshot("L", "x.png")` — layer L, given path
 - `pico.output.screenshot("L", rect)` — layer L, default path, rect
 - `pico.output.screenshot("L", "x.png", rect)` — full form
- Cur layer is preserved (save+restore around the switch).
- **pico.output.sound:** Plays sound.
 - `pico.output.sound (path: string)`
- **pico.output.draw**
 - **pico.output.draw.image:** Draws image.
 - `pico.output.draw.image (path: string, rect: Rect)`
 - **pico.output.draw.layer:** Draws a layer onto the current layer.
 - `pico.output.draw.layer (name: string [, rect: Rect])`
 - `rect` is optional; when omitted (or `nil`), the layer's `scene.dst` is used (full bounds in cur).
 - **pico.output.draw.layers:** Composites the window layer hierarchy (window's children, including world) onto `window.tex`. Implicit in `pico.output.present(true)` (default) and in non-expert mode `auto-present`. Standalone use: compose without mirroring (e.g., to sample composed `window.tex` mid-frame).
 - `pico.output.draw.layers ()`
 - **pico.output.draw.line:** Draws line.
 - `pico.output.draw.line (p1: Pos, p2: Pos)`
 - **pico.output.draw.oval:** Draws oval.
 - `pico.output.draw.oval (rect: Rect)`
 - **pico.output.draw.pixel:** Draws pixel.
 - `pico.output.draw.pixel (pos: Pos)`
 - **pico.output.draw.pixels:** Draws multiple pixels.
 - `pico.output.draw.pixels (ps: {Pos})`
 - **pico.output.draw.pixmap:** Draws a pixmap of colored pixels.
 - `pico.output.draw.pixmap (name: string, pixmap: {{Color}}, rect: Rect)`
 - **pico.output.draw.poly:** Draws polygon.
 - `pico.output.draw.poly (ps: {Pos})`
 - **pico.output.draw.rect:** Draws rectangle.

- `pico.output.draw.rect` (`rect: Rect`)
- **pico.output.draw.text**: Draws text.
 - `pico.output.draw.text` (`text: string, rect: Rect`)
- **pico.output.draw.tri**: Draws triangle.
 - `pico.output.draw.tri` (`p1: Pos, p2: Pos, p3: Pos`)
- **pico.output.draw.video**: Draws video frame.
 - `pico.output.draw.video` (`path: string, rect: Rect`) -> `boolean`
- **pico.pop**: Closes the current scope, freeing every resource created since the matching `pico.push`. Requires the current target layer to be `world` or `window`.
 - `pico.pop` ()
- **pico.push**: Opens a new scope. Resources created until the matching `pico.pop` are released together when it runs.
 - `pico.push` ()
- **pico.quit**: Pushes a quit event to terminate the application.
 - `pico.quit` ()
- **pico.set**: Setters for various properties and settings.
 - omitted table fields keep their current values
 - **pico.set.dim**: Sets both window and world to the same dimensions.
 - `pico.set.dim` (`dim: Dim`)
 - **pico.set.effect**: Sets effect configuration.

```
pico.set.effect {
  alpha: integer,      -- layer opacity 0-255
  color: Color,        -- background (clear) color
  flip: Flip,          -- mirror the layer
  grid: boolean,       -- show the grid aid
  rotate: Rotation,    -- rotation angle + anchor
}
```



- **pico.set.expert**: Toggles expert mode.
 - `pico.set.expert` (`on: boolean [, fps: integer|boolean]`) -> `integer`
 - `fps`: omitted or `false` = wait forever, `true` = as fast as possible, `N>0` = fixed FPS. Explicit `nil` errors.
 - Returns frame period in ms: `-1` = block forever, `0` = immediate, `N>0` = frame period
- **pico.set.layer**: Switches the target layer.
 - `pico.set.layer` (`name: string`) -> `string?`
 - Returns the previous layer name (useful for save/restore).
 - Returns `nil` only on the very first call before any layer has been set

(otherwise returns the previous layer name).

- `name` must be a string; `nil` errors. The two predefined layers are "world" (default) and "window" (expert mode).
- **pico.set.mouse:** Sets mouse cursor position. `pos` is interpreted in `layer`'s frame and projected to window for the SDL warp. `layer` defaults to `cur` when omitted or `nil`.
 - `pico.set.mouse ([layer: string?,] pos: Pos)`
 - `pos.mode`: '!' pixels, '%' percentage, '#' tiles
 - Forms:
 - `pico.set.mouse(pos)` — `cur`
 - `pico.set.mouse(nil, pos)` — explicit `nil` = `cur`
 - `pico.set.mouse('window', pos)` — named layer's frame
- **pico.set.pencil:** Sets pencil configuration.

```
pico.set.pencil {
    color: Color,          -- drawing color
    font:  string,         -- TTF path (false: reset to
built-in)
    style: 'fill'|'stroke', -- shape drawing style
}
```



- **pico.set.scene:** Sets scene configuration.

```
pico.set.scene {
    dim: Dim,          -- texture size (recreates and
clears)
    source: Rect,      -- visible crop of the layer
    clip:  Rect,       -- drawing clip region
    target: Rect,      -- placement in parent (resolved at
each use)
    tile:  Tile,       -- tile size in px (required for '#'
dim)
    clear: boolean,    -- cascade-clear flag
}
```



- **pico.set.video:** Sets video frame.
 - `pico.set.video (name: string, frame: integer) -> boolean`
- **pico.set.window:** Sets window configuration.

```
pico.set.window {
    fullscreen: boolean, -- fullscreen mode
    show:       boolean, -- window visibility
    title:      string,  -- window title
}
```



- **pico.vs**

- Collision checks. Each side has the canonical shape (L_x, v_x) where L_x is the name of a direct child of `cur` (or absent for `cur` itself) and v_x is a `Pos` or `Rect` value relative to L_x .
- Args are matched left-to-right against the 4-slot template $(L1, v1, L2, v2)$. Both explicit `nil`s and trailing omissions are accepted; a mismatched type leaves the slot empty without consuming the arg, so $(L1, L2)$, $(v1, v2)$, $(L1, v2)$, etc. are all valid shortenings.
- When a `rect` slot is absent (both L_x and v_x) it defaults to the bounds of `cur(scene.dst)`. `pos` slots have no default and must always be supplied.
- **pico.vs.pos.pos**: True if two points fall on the same pixel (after integer rounding in `cur`).
 - `pico.vs.pos.pos (L1: string?, p1: Pos, L2: string?, p2: Pos) -> boolean`
 - `p1` and `p2` are required.
- **pico.vs.pos.rect**: True if a point is inside a rectangle.
 - `pico.vs.pos.rect (L1: string?, p1: Pos, L2: string?, r2: Rect?) -> boolean`
 - `p1` is required; `r2` defaults to `cur`'s bounds (or `L2`'s bounds when `L2` is set and `r2` is absent).
- **pico.vs.rect.pos**: True if a point is inside a rectangle (mirror of `pos.rect`).
 - `pico.vs.rect.pos (L1: string?, r1: Rect?, L2: string?, p2: Pos) -> boolean`
 - `p2` is required; `r1` defaults to `cur`'s bounds (or `L1`'s bounds when `L1` is set and `r1` is absent).
- **pico.vs.rect.rect**: True if two rectangles overlap.
 - `pico.vs.rect.rect (L1: string?, r1: Rect?, L2: string?, r2: Rect?) -> boolean`
 - All slots are optional; each missing `rect` side defaults to its layer's bounds (or `cur`'s bounds when the layer is also absent). The 2-string form $(L1, L2)$ compares the bounds of both child layers.

- **pico.xin**

- Composes a child value (`in`) onto a parent `rect` (`out`) and returns a flat value (no parent reference) that resolves to the same absolute coordinates. Mode and anchor of `in` are preserved; numeric `x / y / w / h` are re-expressed within the current scene.
- The module is named `xin` because `in` is a Lua reserved word.
- **pico.xin.rect**: Composes a child `rect` onto a parent `rect`.
 - `pico.xin.rect (out: Rect, in: Rect) -> Rect`

- **pico.xin.pos**: Composes a child position onto a parent rect.
 - `pico.xin.pos (out: Rect, in: Pos) -> Pos`
- **pico.xin.dim**: Composes a child dimension onto a parent rect.
 - `pico.xin.dim (out: Rect, in: Dim) -> Dim`